Mayfly

Aircraft Management & MF700 Documentation for DCSServerBot

Author: Engines

February 2026 — Alpha Release

Mayfly is a new DCSServerBot plugin that brings Royal Navy–style aircraft documentation to DCS World. Named after the Fleet Air Arm's approach to managing individual airframes through their operational lives, it replaces manual spreadsheets and Google Forms with Discord slash commands. Pilots sign out aircraft before a sortie, report defects on return, and engineering staff manage serviceability state—all tracked per–airframe with full audit trails. The system is designed around 892 NAS's existing persistent F-4E Phantom II workflow but is aircraft-type agnostic, supporting any DCS module and any squadron.

1 Motivation

892 Naval Air Squadron currently manages persistent F-4E aircraft using a combination of:

- **Heatblur persistence keys** for aircraft state files
- **Amazon S3** with a Python sync app for sharing keys between pilots
- **Google Sheets** for aircraft logbooks (the "700")
- **Google Forms** for after-action reports
- **Manual staff updates** to reconcile everything after each mission

This process works but is labour-intensive. The WCPO (Watch Chief Petty Officer) must manually cross-reference pilot reports, update the spreadsheet, and communicate aircraft state changes to the squadron. Information lives across multiple platforms with no single source of truth.

Mayfly consolidates this into DCSServerBot, providing a single authoritative system accessible through Discord commands, with automatic event detection for crashes and ejections.

2 The MF700 System

The plugin's documentation model is derived from the MOD Form 700 series—the Royal Navy's aircraft maintenance documentation system. While significantly simplified for the simulation context, it preserves the key concepts that make the real system work.

2.1 F705 — Flight Servicing Certificate

The sign-out/sign-in workflow. A pilot reviews the aircraft's state (limitations, deferred faults, open defects), formally accepts the aircraft, flies the sortie, and signs it back to engineering on return. In Mayfly, this is the `/mayfly signout` and `/mayfly signin` cycle.

2.2 F707 — Aircraft Maintenance Log

The primary defect recording document. Each defect receives a SNOW (Serial Number of Work)—a consecutive number unique to that airframe. Raising an F707 entry places the aircraft unserviceable. It returns to service when all entries are cleared by authorised personnel. Commands: `/mayfly report`, `/mayfly rectify`, `/mayfly defects`.

2.3 F704 — Acceptable Deferred Faults

Not all defects prevent flight. A qualified engineer can defer a fault, assessing the aircraft as still airworthy despite the issue. Deferrals are categorised:

- **CAT A**: As specified (specific time limit)
- **CAT B**: 3 calendar days
- **CAT C**: 10 calendar days

Command: `/mayfly defer`.

2.4 F703 — Limitations Log

Operational limitations that do not make an aircraft unserviceable but restrict its use. Examples: "No night operations", "VMC only", "Max G limited to 4G". Pilots review limitations during sign-out to determine if the aircraft is suitable for the planned mission. Commands: `/mayfly limit`, `/mayfly unlimit`.

3 Aircraft Serviceability

Each aircraft exists in one of three states:

Serviceable
: No open defects. Available for tasking.

Unserviceable
: One or more open F707 entries. Cannot fly until defects are rectified or deferred.

Grounded
: Engineering hold. Aircraft removed from the line regardless of defect state (e.g. structural inspection, modification programme).

Serviceability is determined automatically from the defect log. When a pilot raises a defect via `/mayfly report`, the aircraft becomes unserviceable. When engineering clears or defers all open defects, it returns to serviceable.

Crashes and ejections detected by DCSServerBot's event system automatically mark the aircraft unserviceable, close the flight record, and release the pilot assignment.

4 Commands

All commands are under the `/mayfly` group with autocomplete on squadron, aircraft, and defect selection.

4.1 Pilot Commands

`/mayfly board`
: Fleet status overview—shows all aircraft in a squadron with their current state at a glance.

`/mayfly check`
Detailed status card for a single aircraft: defects, limitations, persistence key, flight hours.
`/mayfly signout`
Sign out an aircraft for a sortie. Locks the aircraft to the pilot.
`/mayfly signin`
Sign the aircraft back in. Record flight hours and any remarks.
`/mayfly report`
Raise a new F707 defect entry against an aircraft.
`/mayfly defects`
View the full F707 defect log for an aircraft.

4.2 Engineering Commands

`/mayfly rectify`
Close a defect—certify the work as complete.
`/mayfly defer`
Defer a defect with a category (CAT A/B/C).
`/mayfly limit`
Add an operational limitation to an aircraft.
`/mayfly unlimit`
Remove a limitation.
`/mayfly ground`
Place an aircraft on engineering hold.
`/mayfly release`
Release a grounded aircraft to the line.

4.3 Admin Commands

`/mayfly add`
Register a new aircraft to a squadron.
`/mayfly edit`
Edit aircraft details (tail number, type, notes, persistence key).
`/mayfly remove`
Write off an aircraft (with reason). Preserves flight records.

5 Architecture

Mayfly is a standard DCSServerBot plugin consisting of:

- **commands.py** — 15 slash commands with autocomplete (∼1,100 lines)
- **listener.py** — Event listener for crash/eject/pilot death
- **db/tables.sql** — 4 PostgreSQL tables with foreign keys to the existing `squadrons` and `players` tables
- **lua/** — Hook stubs for future in-game integration

5.1 Database Schema

mayfly_aircraft
Tail number, type, status, persistence key, squadron FK, flight hours, livery ID
mayfly_flights
Sortie records linking pilot UCID to aircraft with sign-out/sign-in timestamps
mayfly_defects
F707 entries with SNOW numbering, status, deferral category
mayfly_limitations
F703 entries with active/removed state

The schema uses cascading deletes from the `squadrons` table, so removing a squadron cleanly removes all its aircraft, flights, defects, and limitations. Writing off an aircraft preserves it in the database (with timestamp and reason) rather than deleting it.

5.2 Integration Points

- **Userstats plugin** — Provides the `squadrons` and `players` tables that Mayfly references via foreign keys
- **Logbook plugin** — Squadron management commands for creating/editing squadrons
- **MCP API** — All commands are accessible via the REST API for external tooling
- **DCS events** — Crash, ejection, and pilot death events trigger automatic state updates

6 Persistence & the S3 Question

Heatblur's F-4E persistence system is fundamentally *client-side*. The DCS dedicated server does not run module-specific code and cannot access persistence files. This means some form of file synchronisation is required for shared aircraft.

892 NAS currently uses Funky's Python application to sync persistence keys via Amazon S3. The Mayfly plugin manages the *documentation* layer—who has the aircraft, what's wrong with it, how many hours it has flown—while the S3 app handles the *binary state* layer.

The planned approach replaces S3 with a DCS client hook script and PostgreSQL storage:

1. **Storage:** Persistence files (˜2 MB each) stored in the bot's PostgreSQL database, eliminating the need for AWS S3, credentials, or a standalone sync app.
2. **Pre-seed on connect:** A Lua script in the pilot's `Scripts/Hooks/` folder (one-time install, like SRS) downloads *all* squadron cache files when the pilot connects to the server. Files are in place before anyone slots in.
3. **Upload on deslot:** When the pilot unslots or disconnects, the hook uploads *only* the flown aircraft's updated cache file back to the bot API.
4. **Invisible to pilots:** No app to open, no buttons to click. Persistence sync is fully automatic.

Non-persistent aircraft types (Mi-8, Gazelle, etc.) benefit from the documentation system without requiring any file synchronisation.

7 Seed Data

The alpha deployment includes 9 Fleet Air Arm squadrons seeded with authentic serial numbers from Owen Thetford's *British Naval Aircraft since 1912* (6th edition). Each squadron's aircraft are mapped to the closest available DCS module.

Squadron	DCS Type	Historical	A/C
892 NAS	F-4E	Phantom F.G.1	6
800 NAS	AV-8B	Sea Harrier	6
801 NAS	AV-8B	Sea Harrier	6
815 NAS	OH-58D	Lynx H.A.S.3	4
820 NAS	Mi-8	Sea King H.A.S.5	5
845 NAS	Mi-8	Sea King H.C.4	5
846 NAS	Mi-8	Sea King H.C.4	4
849 NAS	Mi-8	Sea King A.E.W.2	3
705 NAS	SA342	Gazelle H.T.2	4
Total			**43**

Serials follow historical allocations. For example, 892 NAS uses the XT-8xx range (the actual Phantom F.G.1 serial block XT857–876), while 800/801 NAS use XZ-4xx and ZA-1xx (Sea Harrier F.R.S.1 blocks XZ450–500 and ZA174–195). Each squadron includes realistic defect and limitation data to demonstrate the documentation workflow.

8 Testing & Next Steps

8.1 Current Status (Alpha)

- All 15 commands implemented and tested via API and Discord
- Deployed on the Hover Stop staging server
- Stanton mission loaded with F-4E aircraft
- Event listener installed for crash/eject detection

8.2 Testing Needed

1. **Signout/signin end-to-end** — Linked pilot signs out an F-4E, flies a sortie on the staging server, signs back in. Verify flight hours are recorded.
2. **Crash event listener** — Sign out an aircraft, crash deliberately, verify the bot automatically marks it unserviceable and closes the flight record.
3. **Defect workflow** — Report a defect after flying, verify it appears in the log, rectify it, confirm aircraft returns to serviceable.

8.3 Roadmap

Beta
Client hook for automated persistence sync. PostgreSQL binary storage with version history. Role-based permissions (pilot vs. engineering vs. admin). Flight hour thresholds for scheduled maintenance triggers.

Release
Upstream PR to DCSServerBot. Documentation. Multi-server support. Historical flight log queries and statistics.

9 For Beta Testers

We are looking for testers who can:

- Fly on the Hover Stop staging server with a linked DCS account
- Test the sign-out/sign-in workflow with F-4E aircraft (892 NAS)
- Try other aircraft types (AV-8B, Mi-8, Gazelle, Kiowa)
- Report defects and exercise the engineering workflow
- Provide feedback on the command UX and documentation model

To get started, connect to the Hover Stop Staging Discord server and run `/mayfly board` to see the fleet. All commands are guild-only and require the DCS role.

Mayfly v0.1 — DCSServerBot Plugin
github.com/engines-wafu/DCSServerBot